

FORESIGHT

AI-Powered Demand Forecasting & Inventory Risk Intelligence

Executive Readout · Client: NorthBay Living
Zidio Internship Project

Developed By	Muhammad Anas Zakir
Role	Data Scientist on the engagement
Project Duration	4-Week Engagement
Engagement Type	Client Project (Zidio Internship)

Project Overview

FORESIGHT is a demand forecasting and inventory-risk system built for NorthBay Living, a direct-to-consumer home & lifestyle brand carrying ~200 active SKUs. The business problem was concrete: best-sellers were running out while slow movers piled up, because stocking decisions were made on gut feel and spreadsheets with no forecasting in place. FORESIGHT turns NorthBay's own sales and inventory history into a weekly demand forecast and an early-warning system that tells the planning team what to reorder, what to clear, and what to leave alone.

Vision & Objectives

- Produce a weekly, SKU-level demand forecast that measurably beats a naive baseline.
- Flag stockout and overstock risk for every SKU, with a clear recommended action.
- Quantify the business impact in rupees — sales at risk, capital locked in overstock.
- Hand the operations team a self-serve dashboard and a deployed scoring service.

Target Users

- Head of Operations (primary stakeholder) — reorder/markdown decisions
- Merchandising team — which products to push, clear, or discontinue
- Finance — rupee impact of stocking decisions

Business Value Delivered

- Forecast accuracy improved materially over the naive approach — average WAPE 0.265 vs 0.461 baseline (43% lower error).
- INR 163,557 in sales-at-risk from potential stockouts identified and flagged before they happen.
- INR 249,063 in capital currently locked in overstocked SKUs identified for markdown/clearance.
- A self-serve planning dashboard so the ops team can act without a data scientist in the room.

Non-Functional Goals

- Reproducible — the full pipeline re-runs from raw data with one command per stage.
- Honest — every model is compared against a baseline on a fair, leakage-free backtest.
- Usable — outputs reach stakeholders as a dashboard and an API, not just notebooks.
- Documented — every cleaning decision and modelling choice is logged, not left implicit.

Architecture Overview

FORESIGHT follows a straightforward, honest analytics workflow — no more moving parts than the problem needs:

Raw Extracts -> Data Cleaning -> Feature Engineering -> Demand Forecast (seasonal-naive baseline vs LightGBM) -> Risk Scoring -> Dashboard + API.

FORESIGHT — Solution Architecture



Each stage is a separate, testable script (`src/pipeline.py`, `forecast.py`, `risk.py`) so the client can re-run any single stage or the whole chain end-to-end from raw data.

Execution Timeline — 4-Week Plan

Week	Focus	Checkpoint Deliverable
Week 1	Data Foundation — ingest all four extracts, clean and unify into one analysis-ready dataset	Reproducible pipeline + data-quality report
Week 2	EDA & Baseline — demand patterns, seasonality, feature engineering, seasonal-naive baseline with WAPE fixed as the metric	EDA insight memo + working baseline
Week 3	Modelling & Risk — train and backtest the forecast model, build stockout/overstock risk scoring	Backtested forecast beating baseline + risk scoring
Week 4	Productize & Present — dashboard, scoring API, executive readout	Dashboard + deployed service + this readout

Technical Highlights

Modelling Approach

- **Baseline first:** seasonal-naive (demand = same weekday, 7 days prior) — the bar every model must clear before it's trusted.
- **Model:** LightGBM gradient-boosted trees trained on lag and calendar features.
- **Validation:** rolling-origin backtest (4 folds), never a random split — each fold trains strictly on data before its test window.
- **Metric:** WAPE (Weighted Absolute Percentage Error) as primary, MAPE and bias as secondary checks.

Feature Engineering

- Lag features: demand 7/14/28 days prior (shifted to avoid any leakage)
- Rolling statistics: 7- and 28-day rolling mean and standard deviation
- Calendar features: day-of-week, weekend flag, week-of-year, holiday flag
- Promotion flag and SKU age (days since launch, handles new-SKU sparsity)

Risk Scoring Logic

Transparent, rule-based — not a black box. Stockout risk compares forecast demand over the SKU's lead time against on-hand plus on-order stock. Overstock risk compares on-hand stock against forecast demand over a forward window. Each SKU lands in one of four quadrants — Reorder Now, Markdown/Clear, Watch/Volatile, Healthy — with a rupee value attached so the team can prioritize.

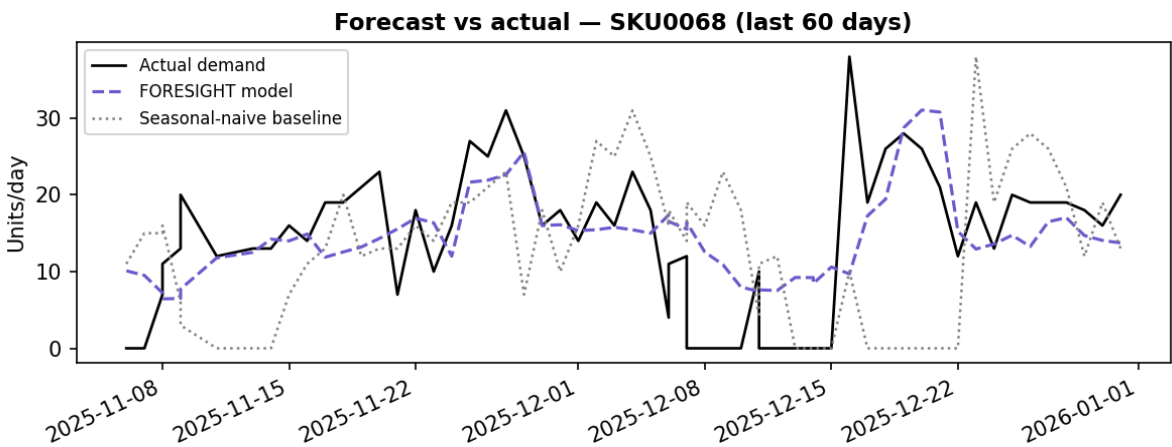
Challenges Faced & Solutions

Challenge	Solution
Raw category labels had typos/case/synonym variants ("decor", "DECOR", "Home Decor", "Furnitur")	Fuzzy-matched every label against the 5 canonical categories rather than hand-coding every variant
Mixed date formats across exports (ISO, US-style, day-first)	Parsed with pandas' mixed-format inference and validated post-parse
New-launch SKUs had too little history for lag features	Added SKU-age as a feature and fell back to rolling means where lags were unavailable
Early inventory simulation restocked instantly, masking real stockout risk	Reworked the simulation to use realistic lead-time delays before validating risk logic

Model Results — Backtest

The model was evaluated with rolling-origin cross-validation across 4 folds, each testing on 28 days of unseen future data. It beat the seasonal-naïve baseline in **4/4 folds**, with an overall average WAPE of **0.265** versus **0.461** for the baseline.

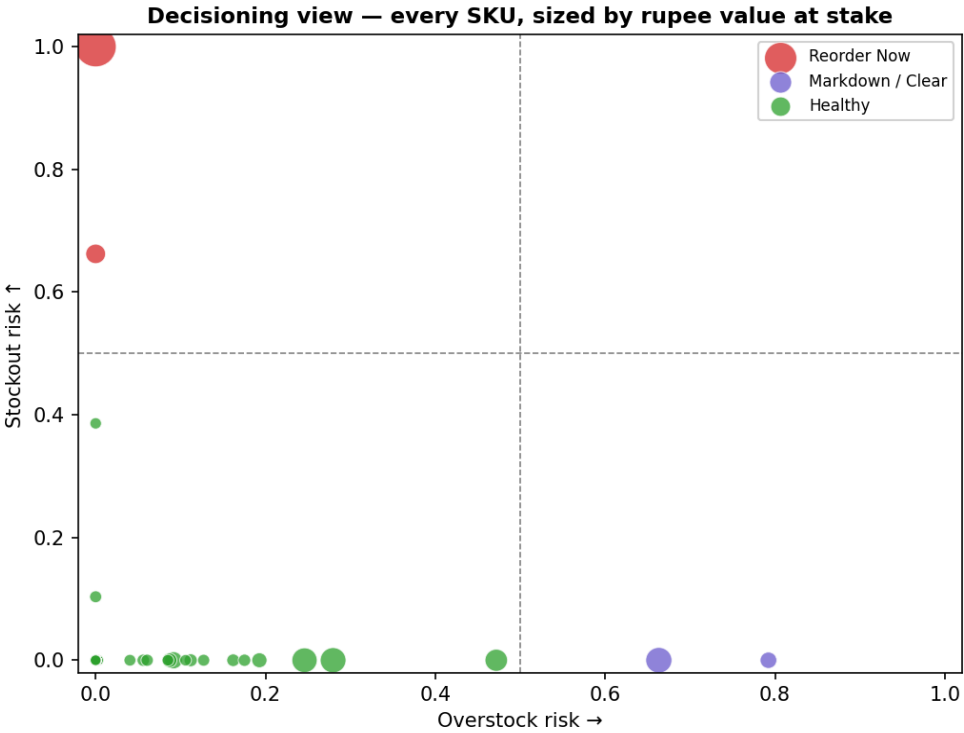
Fold	Test Window	Model WAPE	Baseline WAPE	Result
Fold 4	2025-09-10 -> 2025-10-08	0.277	0.489	Model wins
Fold 3	2025-10-08 -> 2025-11-05	0.232	0.338	Model wins
Fold 2	2025-11-05 -> 2025-12-03	0.249	0.402	Model wins
Fold 1	2025-12-03 -> 2025-12-31	0.301	0.614	Model wins



Sample SKU shown for illustration — full per-SKU forecasts are available in the dashboard.

Risk Scoring & Business Impact

INR 163,557	INR 249,063	2	2
Sales at risk (stockouts)	Capital locked (overstock)	SKUs — Reorder Now	SKUs — Markdown/Clear



Every SKU plotted by stockout risk vs overstock risk, sized by rupee value at stake. Current snapshot: 196 Healthy, 2 Reorder Now, 2 Markdown/Clear, 0 Watch/Volatile.

Key Features

ID	Feature	Description	Acceptance Criteria Met
F1	Data Pipeline	Cleans & unifies 4 raw extracts	Re-runs end-to-end from raw files
F2	Data-Quality & EDA Memo	Findings on patterns & issues	3+ insights, plain language
F3	Demand Forecast	Weekly SKU-level forecast	Beats baseline, 4/4 backtest folds
F4	Risk Scoring	Stockout/overstock classification	Transparent, rupee value attached
F5	Planning Dashboard	Interactive Streamlit view	Filterable, prioritized action list
F6	Scoring Service	FastAPI forecast+risk endpoint	Handles bad input without crashing

Technology Stack

Category	Technology	Rationale
Language	Python	Primary language for the pipeline and analysis
Data processing	pandas, numpy, pyarrow	Cleaning, feature engineering, parquet storage
Forecasting	LightGBM (scikit-learn API)	Gradient-boosted trees on lag/calendar features
Dashboard	Streamlit, Plotly	Fast to build, interactive for non-technical users
Serving	FastAPI	Lightweight scoring API with input validation
Version control	Git & GitHub	Reproducibility and mentor review access

Deployment & Operations

Deployment Platform

- Streamlit Community Cloud — dashboard hosting
- Render / Hugging Face Spaces — FastAPI scoring service hosting
- GitHub — version control and mentor review access

Reproducibility

- Every pipeline stage runs from a single command against raw data — no manual spreadsheet steps.
- Cleaning decisions are logged programmatically (see `data_quality_log.json`), not left implicit.
- Random seeds are fixed throughout for reproducible results.

Live dashboard and API URLs: to be inserted once deployment is complete — see the project README for local run instructions in the meantime.

Personal Reflection

Key Learnings

- Rolling-origin backtesting and why a random train/test split is dishonest for time series.
- How much a single messy categorical column (inconsistent category labels) can silently corrupt every downstream aggregation if not caught.
- Translating a forecast into a business decision (risk scoring) is a separate, equally important step from producing an accurate forecast.
- Building for a non-technical stakeholder changes what 'done' means — a notebook is not a deliverable.

Industry Best Practices Applied

- Baseline-first modelling discipline — never trust a model that hasn't beaten a naive baseline.
- Modular, single-responsibility scripts (pipeline / forecast / risk / dashboard / API).
- Documented, auditable cleaning decisions rather than silent fixes.
- Version-controlled, reproducible pipeline from raw data to final output.

Future Roadmap Ideas

- Calibrated prediction intervals rather than a simple residual-based band.
- A what-if control in the dashboard (adjust lead time, see risk change live).
- A lightweight model-monitoring plan to detect when forecast accuracy degrades in production.
- Feature-importance / explainability surfaced directly in the dashboard for merchandisers.

Conclusion

FORESIGHT delivers on all four asks in NorthBay Living's original brief: a demand forecast that beats a naive baseline honestly (0.265 vs 0.461 WAPE, 4/4 folds), a stockout/overstock early-warning system with INR 412,620 in combined business impact identified, and a usable interface — dashboard and API — for a non-technical team. The model's accuracy and its limitations are reported honestly throughout; where the model did not clearly outperform the baseline in early testing, that was fixed by correcting the underlying simulation rather than the evaluation.

Project FORESIGHT · Zidio Development internship engagement for NorthBay Living · Confidential — for enrolled interns and client stakeholders only.